

마이바티스 CRUD, 동적SQL 정리

오늘의 강의 학습 요약

본 수업은 JDBC의 7단계 연동 흐름과 구조적 한계(예외 처리 강제, 설정/SQL의 자바 코드 내 하드코딩)를 정리한 뒤, 이를 MyBatis가 어떻게 대체하는지로 확장되었습니다. 설정 정보는 properties 파일로, SQL은 Mapper XML로 분리하고, 커넥션 관리는 Pooled 기반으로 처리되며, 세션/메소드 중심의 CRUD 호출로 코드가 간결해지는 흐름을 다뤘습니다.

이어서 MyBatis의 CRUD(Insert/Update/Delete) 실습을 통해 DML의 리턴값 의미, 기본 오토커밋 비활성화로 인한 commit/rollback의 필요성을 확인했습니다. 또한 타입 별칭(Type Alias), XML에서의 비교 연산자 처리(CDATA 및 엔티티), Mapper 다중 구성 시 namespace.id 호출 원칙, 동적 SQL(foreach/where/if/set/choose)과 단일 컬럼 단일값 조회까지 실무형 패턴을 포괄적으로 정리했습니다.

1. JDBC 핵심 흐름과 한계

JDBC는 7단계 절차로 DB를 연동하며, 대부분의 메소드가 `throws SQLException` 형태라 호출 측에서 예외 처리가 필수이고, 설정 정보, SQL이 자바 코드 내부에 있어 변경 시 재컴파일이 필요하다는 구조적 리스크가 있습니다.

JDBC는 전형적으로 드라이버 로딩부터 자원 반납까지 정해진 순서를 따릅니다. 이 흐름은 단일 파일로도 구현되지만, 보통 레이어를 나눠 구성해도 각 단계 자체는 동일합니다.

단계	JDBC 작업	핵심 API/개념
1	DB 연동 4가지 정보 준비	URL, USER, PASSWORD, DRIVER
2	드라이버 로딩	<code>Class.forName(...)</code>
3	커넥션 획득	<code>DriverManager.getConnection(...)</code>
4	SQL 작성	SQL 문자열
5	전송 객체 생성	<code>PreparedStatement</code>
6	SQL 실행	<code>executeUpdate</code> , <code>executeQuery</code>
7	자원 반납	<code>close()</code>

JDBC에서 예외 처리가 필수인 이유는, 주요 메소드들이 `throws SQLException` 로 설계되어 호출자에게 예외가 "위임"되기 때문입니다. 예를 들어 `DriverManager.getConnection(...)` 내부 시그니처가 예외를 던지도록 되어 있어, 호출 코드에서 `try-catch` 또는 `throws` 가 강제됩니다.

JDBC의 또 다른 핵심 리스크는 "설정 정보와 SQL이 자바 코드에 존재"한다는 점입니다. DB 벤더 변경(예: MySQL→Oracle), 계정/URL 변경, SQL 수정이 발생하면 자바 코드 수정 및 재컴파일이 필요하고, 이는 서비스 중단 위험으로 이어질 수 있습니다.

핵심 키워드

#JDBC

#SQLException

#throws

#PreparedStatement

#재컴파일

2. MyBatis로의 전환 구조

MyBatis는 설정 정보는 `properties`, SQL은 Mapper XML로 분리하고, 드라이버 로딩, 커넥션 관리를 프레임워크가 담당하여 JDBC의 예외처리 강제 및 하드코딩 리스크를 완화합니다.

MyBatis는 JDBC의 “4가지 정보”를 자바 코드가 아닌 외부 `properties` 파일로 분리합니다. 이 파일은 키-밸류 형태로 구성되며, 공백/오타가 런타임 연결 실패로 이어질 수 있으므로 값 주변 공백을 특히 주의해야 합니다.

설정 파일(예: `configuration.xml`)에서는 `properties` 를 읽고 `${키}` 형태로 참조합니다. 이때 XML에서 경로 표기는 자바의 패키지 표기(점)가 아니라 슬래시(/)로 해야 합니다.

MyBatis는 JDBC의 드라이버 로딩 및 커넥션 획득 단계를 내부화합니다. 또한 SQL은 Mapper XML로 외부화되며, 자바 코드 수정 없이 SQL만 교체할 수 있어 유지보수성이 좋아집니다.

MyBatis 호출 구조는 “`SQLSessionFactory` → `SqlSession` → CRUD 메소드”로 정리됩니다.

`SqlSessionFactory` 는 세션을 만들어주는 공장이고, `SqlSession` 이 커넥션 역할을 수행합니다.

핵심 키워드

`#properties`

`#Mapper XML`

`#SqlSessionFactory`

`#SqlSession`

`#{}`

3. 커넥션 풀(Pooled)과 close 의미

JDBC는 매 요청마다 커넥션을 생성/해제해 비용이 크며, MyBatis의 `Pooled` 는 커넥션을 미리 확보해 대여/반납 방식으로 재사용하고 `close()` 는 “종료”가 아니라 “반납”으로 동작합니다.

JDBC 코드에서 각 메소드마다 커넥션을 만들고 `close()` 하는 방식은 중복처럼 보이지만, 하나의 커넥션을 전 작업에 공유하는 형태는 동시성/작업 분리 관점에서 위험합니다. 대신 커넥션은 “로컬하게 생성→사용→반납”되는 패턴이 안전합니다.

다만 커넥션 생성은 네트워크 경로 탐색 등 비용이 커서 전체 수행 시간 대부분이 커넥션 획득에 쓰일 수 있습니다. 이를 줄이기 위해 커넥션 풀은 커넥션을 “미리” 여러 개 만들어 저장해두고, 필요 시 빌려주고 사용 후 반환받습니다.

항목	JDBC 단순 방식	커넥션 풀(Pooled) 방식
커넥션 생성	작업마다 생성합니다.	초기/종설 시 생성합니다.
close 의미	실제 연결을 끊습니다.	풀에 반납합니다.
성능 특성	커넥션 비용이 누적됩니다.	재사용으로 비용이 감소합니다.
누락 위험	누락 시 리소스 누수입니다.	누락 시 풀 고갈로 장애가 됩니다.

풀 환경에서 `close()` 를 하지 않으면 “빌려간 커넥션을 반환하지 않는” 상태가 되어, 결국 풀의 커넥션이 소모되어 다음 요청이 커넥션을 받지 못하고 에러가 발생합니다. 따라서 MyBatis에서도 세션 `close()` 는 필수입니다.

핵심 키워드 `#Pooled` `#커넥션 풀` `#close 반납` `#풀 고갈` `#DataSource`

4. Mapper XML 기본 문법과 바인딩 변수

MyBatis Mapper의 `select/insert/update/delete` 는 `id` 가 필수이며, 파라미터 바인딩은 `#{}` 를 사용하고 `${}` 는 사용 가능하지만 보안, 안정성 측면에서 지양합니다.

Mapper의 `select` 는 파라미터 유무에 따라 `parameterType` 이 있을 수도 없을 수도 있습니다.

`resultType` 은 조회 결과 레코드를 담은 타입이며 DTO뿐 아니라 `HashMap` 등도 사용할 수 있습니다.

바인딩 변수는 JDBC의 `?` 가 MyBatis에서는 `#{}` 로 바뀐 개념입니다. 반면 `${}` 는 문자열 치환 형태로 동작해 의도치 않은 SQL 조합 및 보안 이슈(인젝션 등)를 유발할 수 있으므로 지양하는 것이 안전합니다.

레거시 환경에서 iBATIS를 접하면 바인딩 표현이 `#변수#` 형태로 보일 수 있으며, 이는 구버전 문법으로 이해해야 합니다.

핵심 키워드

`#id`

`#resultType`

`#parameterType`

`##{}`

`#{}`

5. CRUD 실습: Insert, Update, Delete와 커밋

MyBatis의 DML(insert/update/delete)은 리턴 타입이 수정된 행 수를 의미하는 `int` 이며, 기본 이 오토커밋이 아니므로 `commit()` 을 호출해야 실제 DB에 반영됩니다.

MyBatis에서 `session.insert(...)`, `session.update(...)`, `session.delete(...)` 는 모두 "문자열 id만 받는 형태"와 "id + 파라미터 객체를 받는 형태" 두 오버로드를 가집니다. 파라미터가 존재하면 두 번째 형태를 사용합니다.

DML 리턴값은 JDBC의 `executeUpdate()` 결과와 동일하게 "영향 받은 행(row) 수"입니다. 예를 들어 insert 성공 시 보통 1이 반환됩니다.

MyBatis는 JDBC와 달리 기본 오토커밋이 아니므로, insert 성공 메시지를 출력해도 `commit()` 을 하지 않으면 DB에 반영되지 않을 수 있습니다. 또한 실패 시 `rollback()` 으로 트랜잭션을 되돌릴 수 있습니다.

작업	호출 메소드	리턴값 의미	트랜잭션
Insert	<code>session.insert(id, param)</code>	저장된 행 수입니다.	<code>commit()</code> 필요합니다.
Update	<code>session.update(id, param)</code>	수정된 행 수입니다.	<code>commit()</code> 필요합니다.
Delete	<code>session.delete(id, param)</code>	삭제된 행 수입니다.	<code>commit()</code> 필요합니다.

핵심 키워드

`#insert`

`#update`

`#delete`

`#commit`

`#rollback`

6. 타입 별칭(Type Alias) 등록

MyBatis는 기본 별칭과 사용자 정의 별칭을 제공하며, 별칭은 Configuration XML의 `typeAliases` 에서 클래스 단위 또는 패키지 단위로 등록할 수 있습니다.

기본형/컬렉션 등은 이미 별칭으로 제공되며(대부분 소문자), `typeAliases` 섹션에서 확인할 수 있습니다. 사용자 DTO도 별칭 등록을 통해 Mapper에서 패키지 포함 FQCN을 반복해서 쓰지 않도록 단순화할 수 있습니다.

별칭 등록은 두 가지가 핵심입니다. 첫째, 클래스 단위로 `<typeAlias alias="..." type="..."/>` 를 등록하는 방식이 있습니다. 둘째, `<package name="..."/>` 로 패키지 스캔 후 DTO 클래스에 `@Alias` 로 별칭을 주는 방식이 있습니다.

Configuration XML에서는 태그의 “작성 순서”가 중요할 수 있으며, `typeAliases` 는 `properties` 이후, `environments` 이전 위치에 배치하는 규칙을 따릅니다.

핵심 키워드 `#typeAliases` `#typeAlias` `#package` `#@Alias` `#별칭`

7. XML 비교 연산자와 CDATA

Mapper SQL에서 `<` `>` 는 XML 태그로 오인될 수 있어 `<` `>` 엔티티 또는 `<![CDATA[ ... ]]>` 로 처리해야 하며, 실무에서는 CDATA로 SQL 전체를 감싸는 방식이 일반적입니다.

XML 기반 Mapper에서 비교 연산자 `<` `>` 를 그대로 쓰면 파서가 태그로 해석해 에러가 발생할 수 있습니다. 이를 회피하는 방법은 두 가지입니다.

첫 번째는 엔티티로 치환하는 방식이며, `<` 는 `<` , `>` 는 `>` 로 씁니다. 두 번째는 SQL 구문을 CDATA 섹션으로 감싸 “파서가 해석하지 않고 문자 데이터로 처리”하도록 만드는 방식입니다.

CDATA는 XML 문법이며 MyBatis 전용이 아닙니다. SQL을 안전하게 그대로 유지해야 할 때 유용합니다.

핵심 키워드 `#CDATA` `#<` `#>` `#XML 파서` `#비교연산자`

8. Mapper 다중 구성과 Namespace 호출

Mapper가 여러 개일 때 `id` 는 전체 Mapper 범위에서 충돌할 수 있으므로 `namespace.id` 형태로 호출하는 것이 표준이며, namespace를 분리하면 동일 `id`를 재사용해도 구분됩니다.

Mapper 파일을 여러 개 만들면 Configuration에 각각 등록해야 해당 SQL을 호출할 수 있습니다. 또한 `id` 가 파일이 다르더라도 충돌이 날 수 있으며, 이는 “어떤 statement를 호출할지 모호”해지는 원인입니다.

이 문제를 해결하는 핵심 개념이 `namespace` 입니다. 자바에서 동일 클래스명이 패키지로 구분되듯, MyBatis는 동일 `id` 라도 `namespace` 가 다르면 구분됩니다.

호출 시에는 `selectOne("namespace.id", param)` 처럼 네임스페이스를 반드시 붙이는 습관이 권장됩니다. 이는 `id` 관리 부담을 줄이고, 테이블별 CRUD에서 동일한 `id`를 통일해 사용할 수 있게 해줍니다.

핵심 키워드

`#namespace`

`#statement id`

`#중복 충돌`

`#namespace.id`

`#mapper 등록`

9. 자주 만나는 런타임 에러와 진단

`Mapped Statements collection does not contain value for ...` 에러는 Mapper의 `id` (또는 `namespace.id`)가 등록/일치하지 않을 때 발생하며, 런타임에 터지므로 복사-붙여넣기로 정확히 맞추는 습관이 중요합니다.

MyBatis는 SQL statement를 문자열 키로 찾기 때문에, 컴파일 타임이 아니라 실행 시점에 `id` 오타가 에러로 드러납니다. 따라서 `id` 타이핑보다는 복사-붙여넣기를 권장하며, 특히 네임스페이스까지 포함해 호출하는 습관이 디버깅 비용을 줄입니다.

핵심 키워드

`#Mapped Statements`

`#런타임 에러`

`#id 오타`

`#복사-붙여넣기`

`#namespace`

10. 환경 구성: EMP 예제로 확장

EMP 테이블 기반으로 프로젝트를 확장하며 DTO 생성, properties/Configuration/Mapper 구성, Eclipse properties 인코딩(UTF-8) 기본 설정 변경까지 실무에 필요한 초기 세팅을 정리합니다.

EMP 예제에서는 DTO를 테이블 컬럼에 맞춰 구성하고, 생성자/세터/ toString() 등을 준비합니다. 낱짜 컬럼은 가공 목적이 없다면 문자열로 받아 단순 출력하는 방식도 선택할 수 있습니다.

properties 파일은 반복적으로 UTF-8 설정이 필요하므로, Eclipse에서 Content Types 설정을 통해 .properties 기본 인코딩을 UTF-8로 바꾸는 방식이 소개되었습니다. 이는 팀 개발에서 한글 깨짐을 줄이는 실무 팁입니다.

구성 요소	역할	주의점
DTO	레코드 저장용 객체입니다.	컬럼 타입 매핑을 명확히 합니다.
jdbc.properties	4가지 연결 정보를 외부화합니다.	공백/오타를 특히 주의합니다.
configuration.xml	properties, aliases, environments, mappers를 묶습니다.	태그 순서를 지킵니다.
EMPMapper.xml	SQL 정의 파일입니다.	namespace를 명확히 둡니다.

핵심 키워드

#EMP DTO

#properties UTF-8

#configuration.xml

#EMPMapper

#초기설정

11. 동적 SQL: foreach로 멀티 Insert

`foreach` 는 컬렉션을 반복해 SQL 조각을 생성하며, 멀티 insert에서는 (...) 구문을 반복하고 `separator=","` 로 심표를 자동 삽입합니다.

멀티 insert는 전달된 DTO 리스트 개수만큼 values 튜플을 생성해야 합니다. 자바에서는 `List<EmpDTO>` 를 만들어 `session.insert("namespace.insertMulti", list)` 형태로 전달합니다.

Mapper에서는 `parameterType` 을 `arraylist` 또는 `list` 별칭으로 두고, `foreach` 에서 `collection` 에 리스트를 지정합니다. `item` 은 반복 중 꺼낸 요소(여기서는 DTO)이며, `#{item.empno}` 처럼 DTO 필드를 참조합니다.

```
<insert id="insertMulti" parameterType="arraylist">
  INSERT INTO EMP (EMPNO, ENAME, SAL)
  VALUES
  <foreach collection="list" item="dto" separator=",">
    (#{dto.empno}, #{dto.ename}, #{dto.sal})
  </foreach>
</insert>
```

핵심 키워드

`#foreach`

`#separator`

`#collection`

`#item`

`#멀티 insert`

12. 동적 SQL: foreach로 멀티 Delete, Select(IN)

IN 조건의 멀티 삭제/조회는 `foreach` 로 값을 나열하며, `open` 과 `close` 로 괄호를 자동 생성하고, 값이 비어있을 가능성이 있으면 `where` 태그로 안전하게 구성합니다.

멀티 delete는 `DELETE FROM EMP WHERE EMPNO IN (...)` 형태가 됩니다. 값 목록은 리스트로 전달하며 `foreach` 에서 쉼표 구분으로 출력합니다.

`open` 은 반복 전에 한 번 삽입되는 접두 문자열이고, `close` 는 반복이 끝난 후 한 번 삽입되는 접미 문자열입니다. 이를 사용하면 괄호를 태그 밖에서 직접 타이핑하지 않아도 됩니다.

리스트가 비어 `foreach` 가 한 번도 돌지 않으면 `IN` (만 남는 등 문법 오류가 날 수 있습니다. 이 위험을 회피하기 위해 `where` 태그를 사용해 조건이 비어 있을 때 `WHERE` 자체가 생성되지 않도록 설계할 수 있습니다.

핵심 키워드

`#IN`

`#open`

`#close`

`#where`

`#멀티 delete`

13. 동적 SQL: if로 조건 1개/여러 개 처리

`if test="..."` 는 단일 조건 블록을 제공하며, 여러 조건을 조합할 때는 각 조건 앞에 `AND/OR` 를 두고 `where` 태그가 선행 논리연산자를 자동 정리해주는 패턴이 표준입니다.

MyBatis의 `if` 는 자바의 `if-else if-else` 처럼 블록 체인이 아니라, "조건별로 SQL 조각을 붙이거나 생략"하는 방식입니다. 따라서 여러 조건이 있을 때는 각각을 `if` 로 나누고, 조합 연산자(`OR` , `AND`)를 두 번째 조건부터 앞에 붙이는 형태가 자연스럽습니다.

`where` 태그는 조건이 하나도 생성되지 않으면 `WHERE` 자체를 만들지 않고, 조건이 생성되면 앞쪽의 불필요한 `AND/OR` 를 제거해줍니다. 이 성질을 활용하면 "조건이 하나만 있을 때"도, "둘 다 있을 때"도 안전한 SQL이 생성됩니다.

숫자 타입(예: salary)은 null 비교 대신 "0인지 여부" 등으로 조건을 설계해야 하며, DTO에서 기본값을 어떻게 둘지도 함께 고려해야 합니다.

핵심 키워드

`#if`

`#test`

`#where`

`#AND/OR` 정리

`#조건부 조회`

14. 동적 SQL: set으로 조건부 Update

동적 update는 조건에 따라 SET 절의 컬럼이 달라지므로, 실행 처리 문제를 해결하기 위해 `set` 태그를 사용하며, `set` 은 불필요한 후행 콤마를 자동 제거합니다.

update에서 일부 컬럼만 수정할 때, 조건에 따라 SET 안의 항목이 생성되거나 사라지며 마지막 실행이 남아 문법 오류가 발생하기 쉽습니다. 이를 해결하는 대표 태그가 `set` 입니다.

구성은 "항상 수정되는 컬럼"과 "조건부 수정 컬럼"을 나누고, 조건부 컬럼은 `if` 로 감쌉니다. `set` 태그가 마지막 콤마를 자동으로 제거하므로 각 라인 끝에 콤마를 붙이는 패턴이 안전해집니다.

핵심 키워드

`#set`

`#동적 update`

`#콤마 제거`

`#if`

`#SET 절`

15. 동적 SQL: choose/when/otherwise

`choose` 는 SQL의 CASE 또는 자바의 switch에 가까운 분기 구조로, 여러 조건 중 하나를 선택해서 다른 SQL 조각을 생성할 때 사용합니다.

`choose` 는 여러 `when` 중 최초로 참이 되는 블록을 택하고, 어떤 조건도 만족하지 않으면 `otherwise` 를 적용합니다. 예를 들어 job 값이 SALESMAN 이면 salary 기준을 1500으로, CLERK 이면 2500으로, 그 외는 3000으로 조회하는 식의 정책 분기에 적합합니다.

`test` 속성은 XML 속성이므로 큰따옴표로 감싸고, 문자열 비교 값은 작은따옴표로 감싸는 형태를 지켜야 합니다. 비교 연산자를 쓰는 구문은 CDATA로 감싸는 습관이 SQL 유지에 유리합니다.

핵심 키워드

`#choose`

`#when`

`#otherwise`

`#switch`

`#CASE`

16. 단일 컬럼 단일값 조회(selectOne)

COUNT, MAX, MIN, SUM 같은 집계 결과처럼 “컬럼 값 하나”만 반환되는 조회는 `selectOne` 으로 받고, `resultType` 은 `int` 나 `string` 등 단일 스칼라 타입으로 지정합니다.

기존 실습은 DTO(레코드) 단위 반환 중심이었지만, 단일값 조회는 반환 타입이 DTO가 아니라 스칼라가 됩니다. 예를 들어 `SELECT COUNT(*) FROM EMP` 는 결과가 정수 하나이므로 `selectOne` 으로 받아 `int` 로 처리합니다.

파라미터가 없는 경우 `selectOne(id)` 형태로 호출하며, 조건이 있으면 `selectOne(id, param)` 형태로 호출합니다. `resultType` 을 지정하지 않으면 매핑 오류가 날 수 있으므로 스칼라 타입을 명시하는 것이 안전합니다.

핵심 키워드

`#selectOne`

`#COUNT`

`#스칼라 매핑`

`#resultType`

`#집계함수`